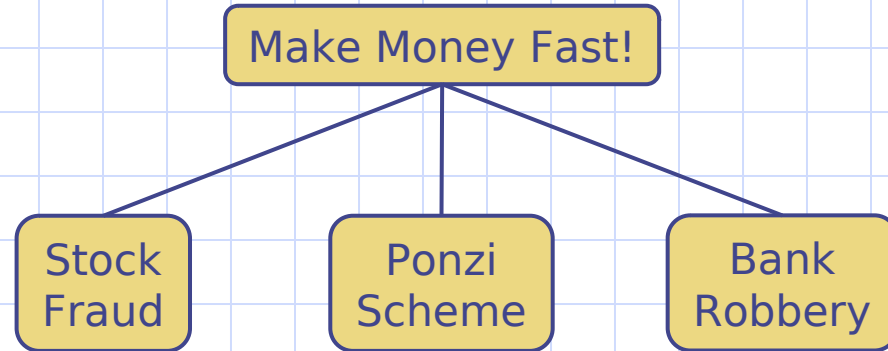
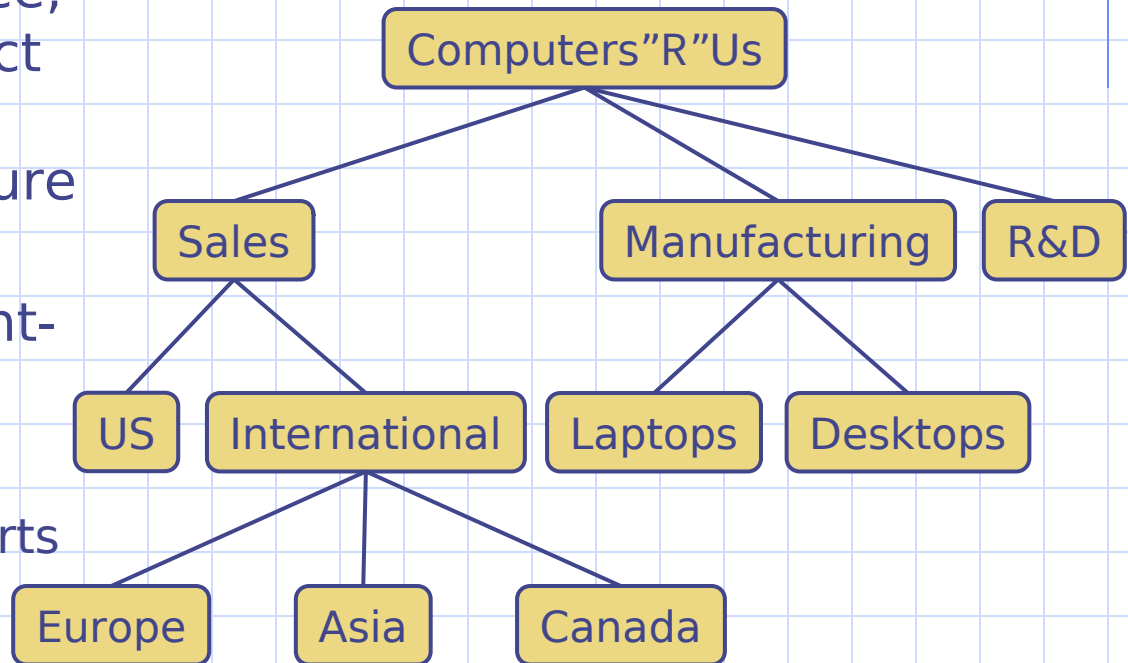


Trees



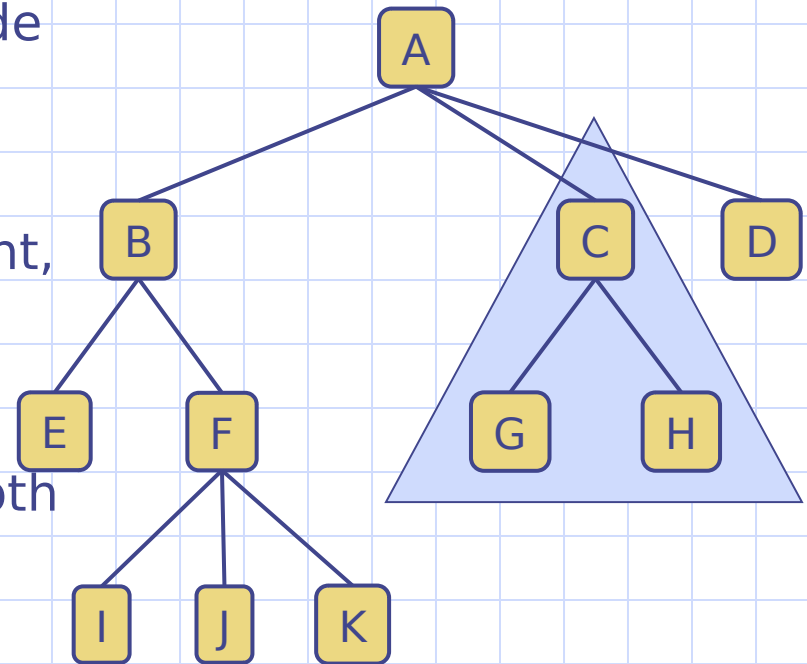
What is a Tree

- ❑ In computer science, a tree is an abstract model of a hierarchical structure
- ❑ A tree consists of nodes with a parent-child relation
- ❑ Applications:
 - Organization charts
 - File systems
 - Programming environments



Tree Terminology

- Root: node without parent (A)
- Internal node: node with at least one child (A, B, C, F)
- External node (a.k.a. leaf): node without children (E, I, J, K, G, H, D)
- Ancestors of a node: parent, grandparent, grand-grandparent, etc.
- Depth of a node: number of ancestors
- Height of a tree: maximum depth of any node (3)
- Descendant of a node: child, grandchild, grand-grandchild, etc.
- Subtree: tree consisting of a node and its descendants



Tree ADT

- We use positions to abstract nodes
- Generic methods:
 - Integer `len()`
 - Boolean `is_empty()`
 - Iterator `positions()`
 - Iterator `iter()`
- Accessor methods:
 - position `root()`
 - position `parent(p)`
 - Iterator `children(p)`
 - Integer `num_children(p)`
- ◆ Query methods:
 - Boolean `is_leaf(p)`
 - Boolean `is_root(p)`
- ◆ Update method:
 - element `replace (p, o)`
- ◆ Additional update methods may be defined by data structures implementing the Tree ADT

Python

```
1 class Tree:
2     """Abstract base class representing a tree structure."""
3
4     #----- nested Position class -----
5     class Position:
6         """An abstraction representing the location of a single element."""
7
8         def element(self):
9             """Return the element stored at this Position."""
10            raise NotImplementedError('must be implemented by subclass')
11
12        def __eq__(self, other):
13            """Return True if other Position represents the same location."""
14            raise NotImplementedError('must be implemented by subclass')
15
16        def __ne__(self, other):
17            """Return True if other does not represent the same location."""
18            return not (self == other)          # opposite of __eq__
19
```

```
20 # ----- abstract methods that concrete subclass must support -----
21 def root(self):
22     """Return Position representing the tree's root (or None if empty)."""
23     raise NotImplementedError('must be implemented by subclass')
24
25 def parent(self, p):
26     """Return Position representing p's parent (or None if p is root)."""
27     raise NotImplementedError('must be implemented by subclass')
28
29 def num_children(self, p):
30     """Return the number of children that Position p has."""
31     raise NotImplementedError('must be implemented by subclass')
32
33 def children(self, p):
34     """Generate an iteration of Positions representing p's children."""
35     raise NotImplementedError('must be implemented by subclass')
36
37 def __len__(self):
38     """Return the total number of elements in the tree."""
39     raise NotImplementedError('must be implemented by subclass')

```

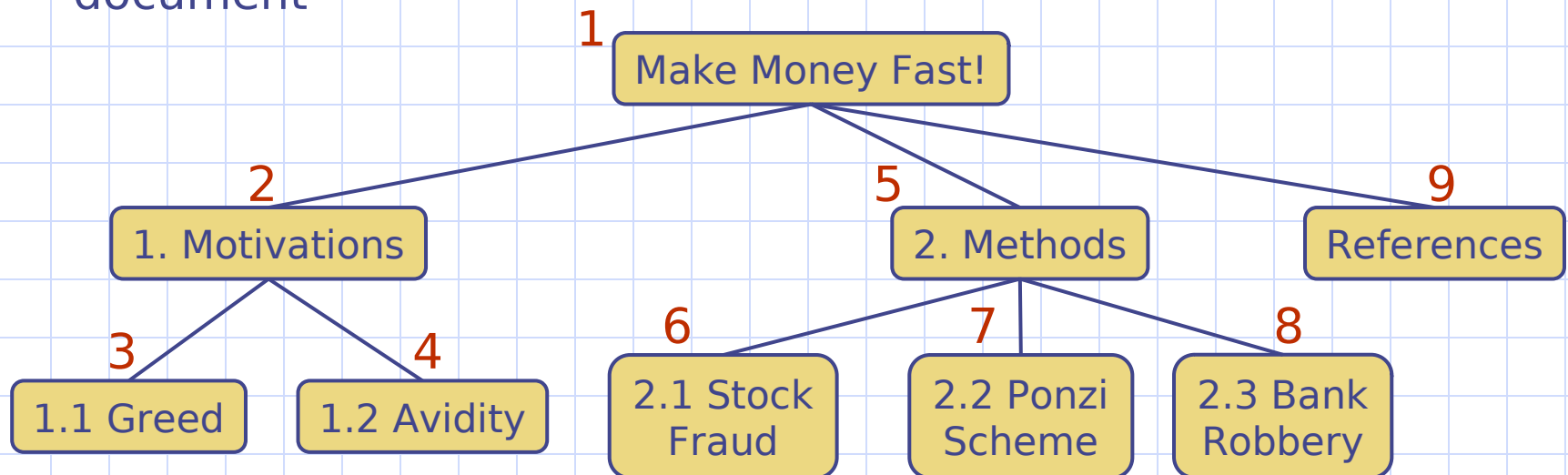
```
40 # ----- concrete methods implemented in this class -----
41 def is_root(self, p):
42     """Return True if Position p represents the root of the tree."""
43     return self.root() == p
44
45 def is_leaf(self, p):
46     """Return True if Position p does not have any children."""
47     return self.num_children(p) == 0
48
49 def is_empty(self):
50     """Return True if the tree is empty."""
51     return len(self) == 0

```

Preorder Traversal

- A traversal visits the nodes of a tree in a systematic manner
- In a preorder traversal, a node is visited before its descendants
- Application: print a structured document

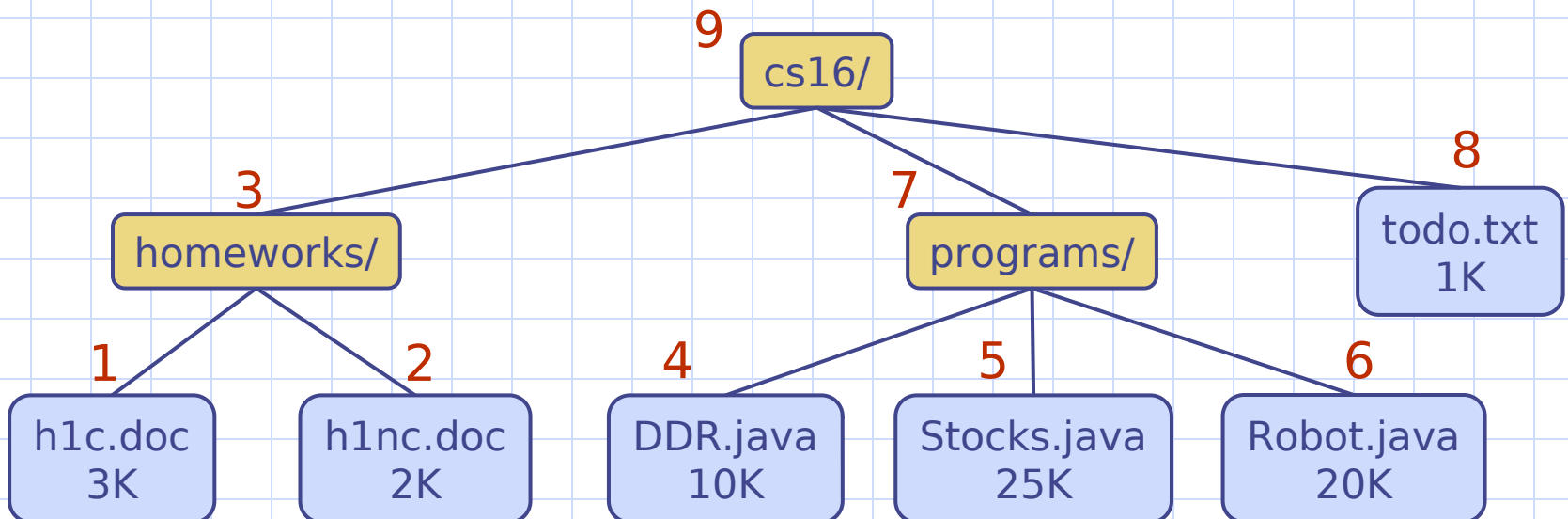
Algorithm *preOrder(v)*
visit(v)
for each child *w* of *v*
preorder(w)



Postorder Traversal

- In a postorder traversal, a node is visited after its descendants
- Application: compute space used by files in a directory and its subdirectories

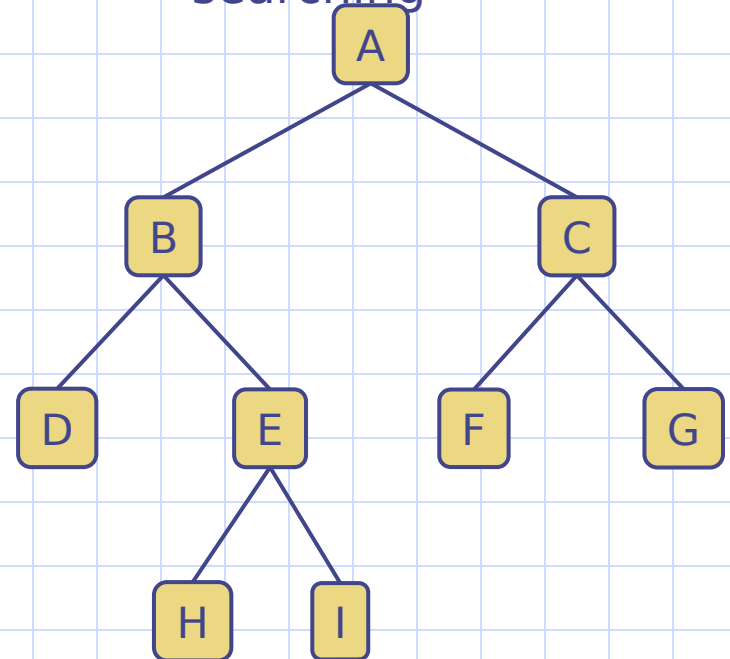
Algorithm *postOrder(v)*
for each child *w* of *v*
 postOrder(w)
visit(v)



Binary Trees

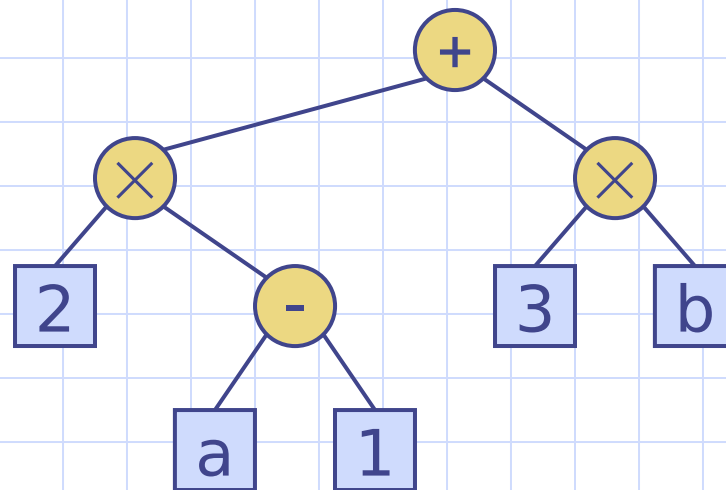
- A binary tree is a tree with the following properties:
 - Each internal node has at most two children (exactly two for **proper** binary trees)
 - The children of a node are an ordered pair
- We call the children of an internal node **left child** and **right child**
- Alternative recursive definition: a binary tree is either
 - a tree consisting of a single node, or
 - a tree whose root has an ordered pair of children, each of which is a binary tree

- Applications:
 - arithmetic expressions
 - decision processes
 - searching



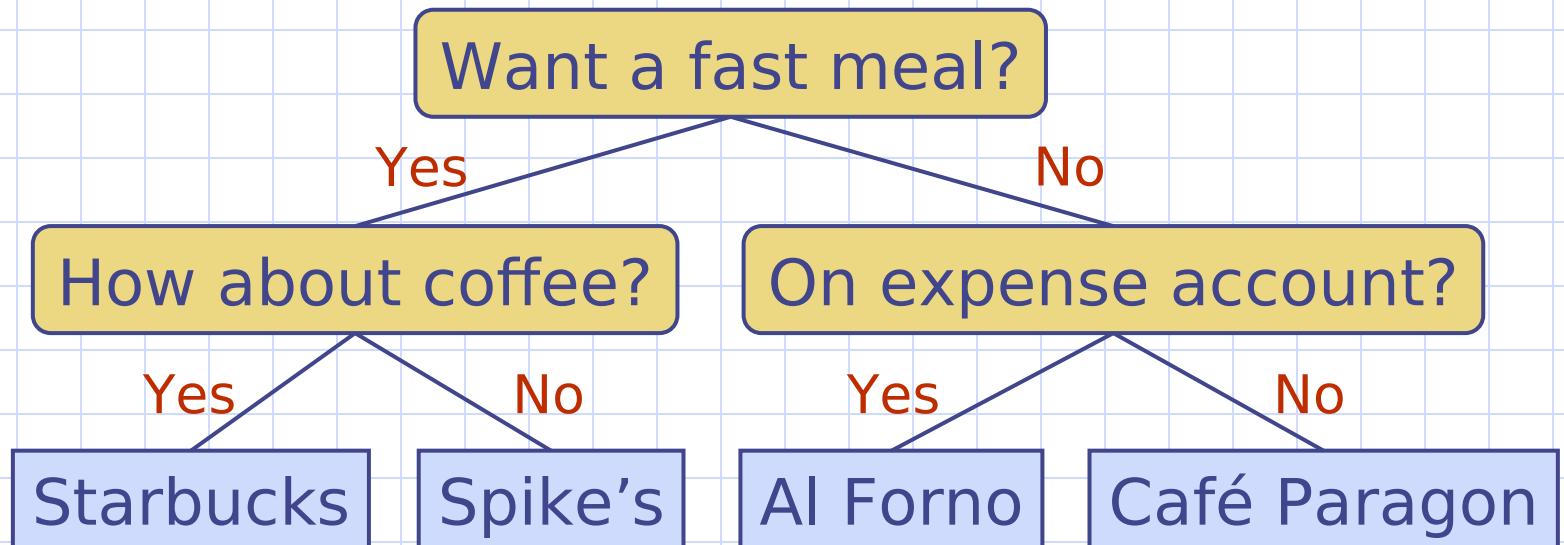
Arithmetic Expression Tree

- Binary tree associated with an arithmetic expression
 - internal nodes: operators
 - external nodes: operands
- Example: arithmetic expression tree for the expression $(2 \times (a - 1) + (3 \times b))$



Decision Tree

- Binary tree associated with a decision process
 - internal nodes: questions with yes/no answer
 - external nodes: decisions
- Example: dining decision



Properties of Proper Binary Trees

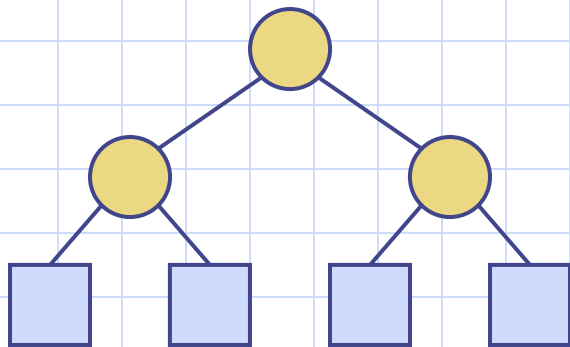
□ Notation

n number of nodes

e number of external nodes

i number of internal nodes

h height



◆ Properties:

- $e = i + 1$

- $n = 2e - 1$

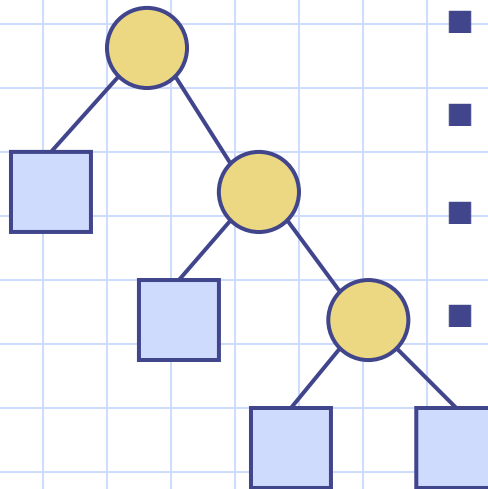
- $h \leq i$

- $h \leq (n - 1)/2$

- $e \leq 2^h$

- $h \geq \log_2 e$

- $h \geq \log_2 (n + 1) - 1$



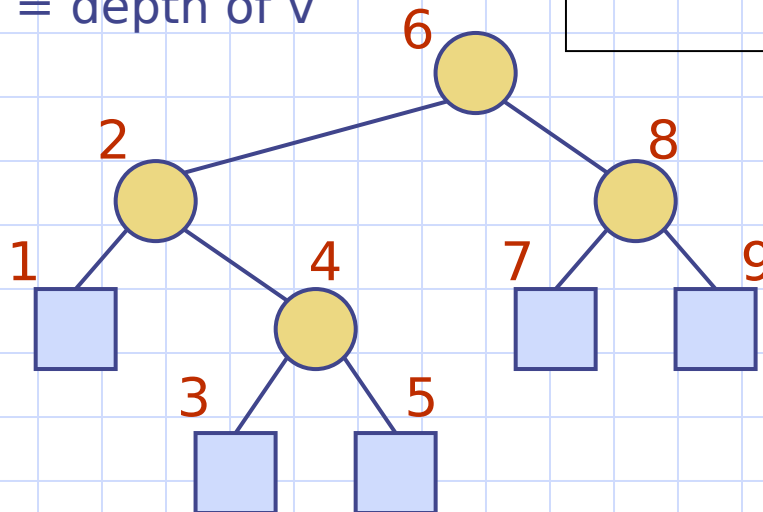
BinaryTree ADT

- The BinaryTree ADT extends the Tree ADT, i.e., it inherits all the methods of the Tree ADT
- Additional methods:
 - position **left**(p)
 - position **right**(p)
 - position **sibling**(p)
- Update methods may be defined by data structures implementing the BinaryTree ADT

Inorder Traversal

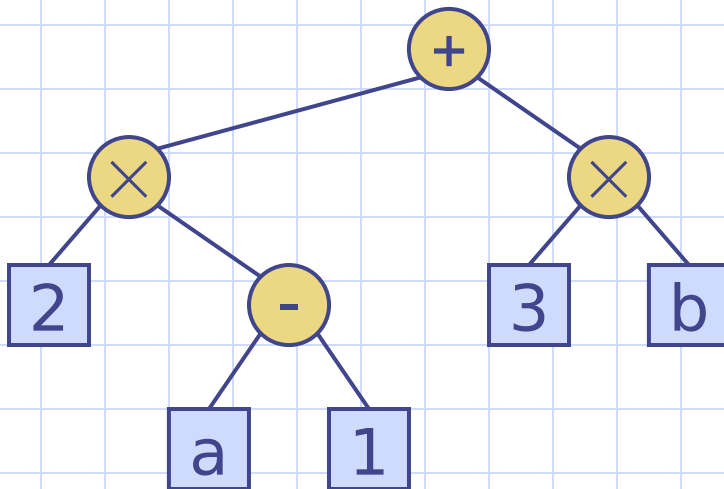
- In an inorder traversal a node is visited after its left subtree and before its right subtree
- Application: draw a binary tree
 - $x(v)$ = inorder rank of v
 - $y(v)$ = depth of v

```
Algorithm inOrder( $v$ )  
  if  $v$  has a left child  
    inOrder (left ( $v$ ))  
  visit( $v$ )  
  if  $v$  has a right child  
    inOrder (right ( $v$ ))
```



Print Arithmetic Expressions

- Specialization of an inorder traversal
 - print operand or operator when visiting node
 - print "(" before traversing left subtree
 - print ")" after traversing right subtree

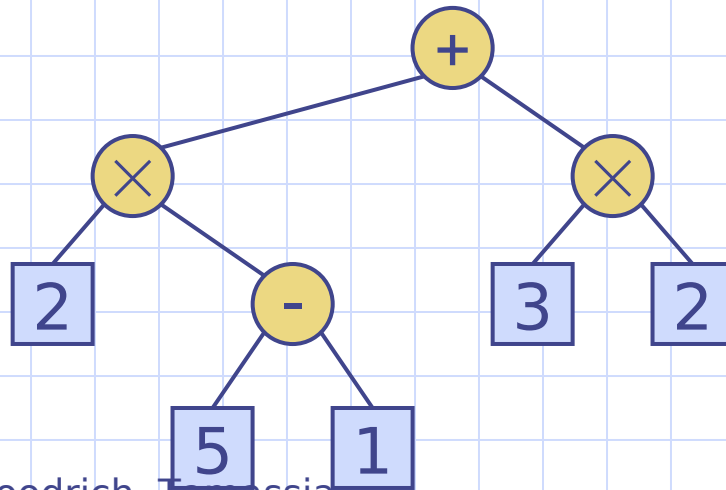


Algorithm *printExpression(v)*
if *v* **has a left child**
 print("(")
 inOrder (*left*(*v*))
 print(*v.element* ())
if *v* **has a right child**
 inOrder (*right*(*v*))
 print(")")

$((2 \times (a - 1)) + (3 \times b))$

Evaluate Arithmetic Expressions

- Specialization of a postorder traversal
 - recursive method returning the value of a subtree
 - when visiting an internal node, combine the values of the subtrees



Algorithm *evalExpr(v)*

if *is_leaf*(v)

return *v.element*()

else

$x \leftarrow \text{evalExpr}(\text{left}(v))$

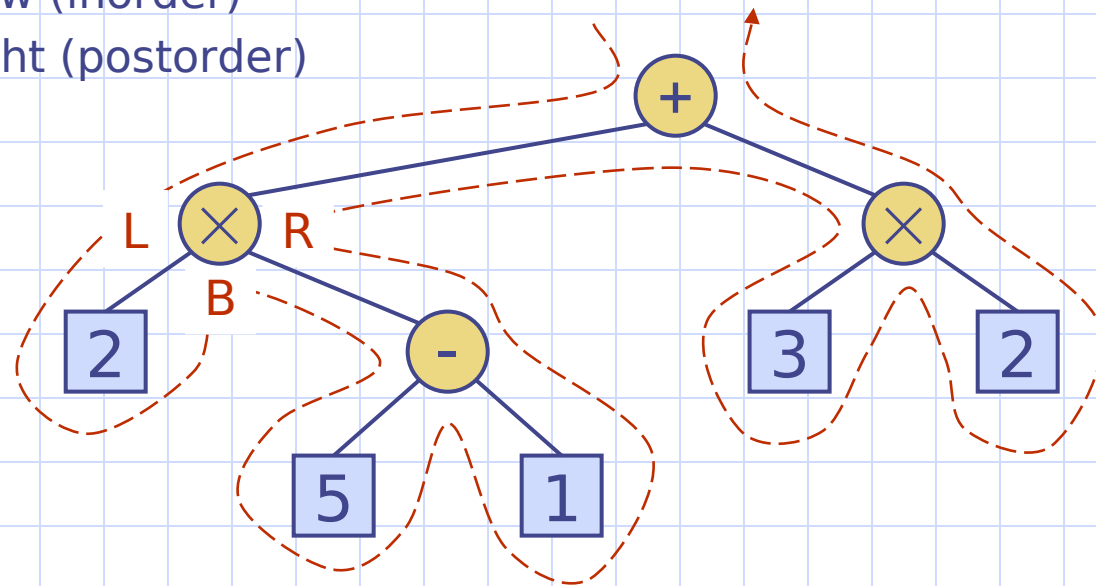
$y \leftarrow \text{evalExpr}(\text{right}(v))$

$\diamond \leftarrow$ operator stored at *v*

return $x \diamond y$

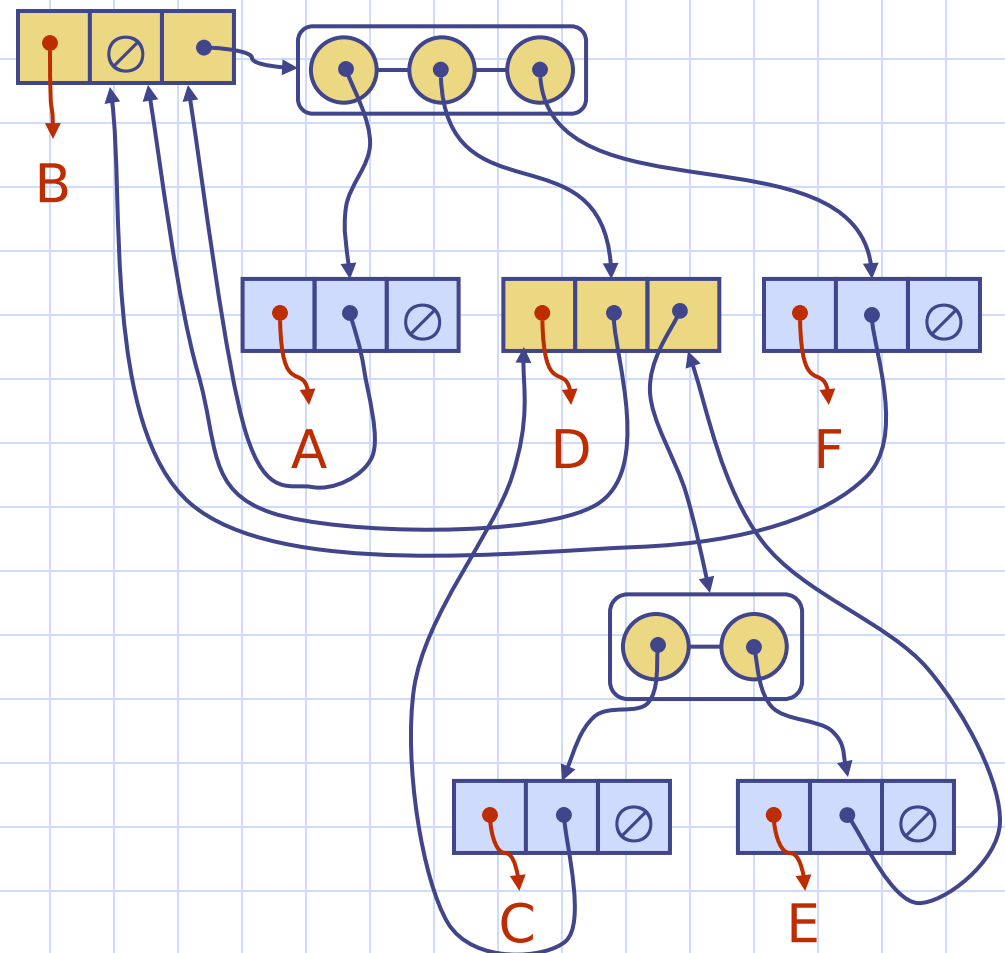
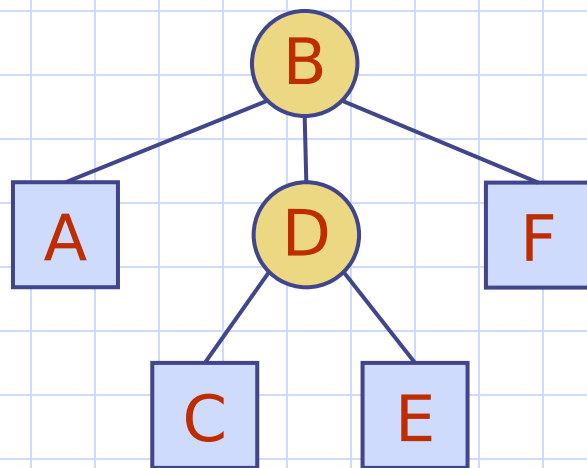
Euler Tour Traversal

- Generic traversal of a binary tree
- Includes a special cases the preorder, postorder and inorder traversals
- Walk around the tree and visit each node three times:
 - on the left (preorder)
 - from below (inorder)
 - on the right (postorder)



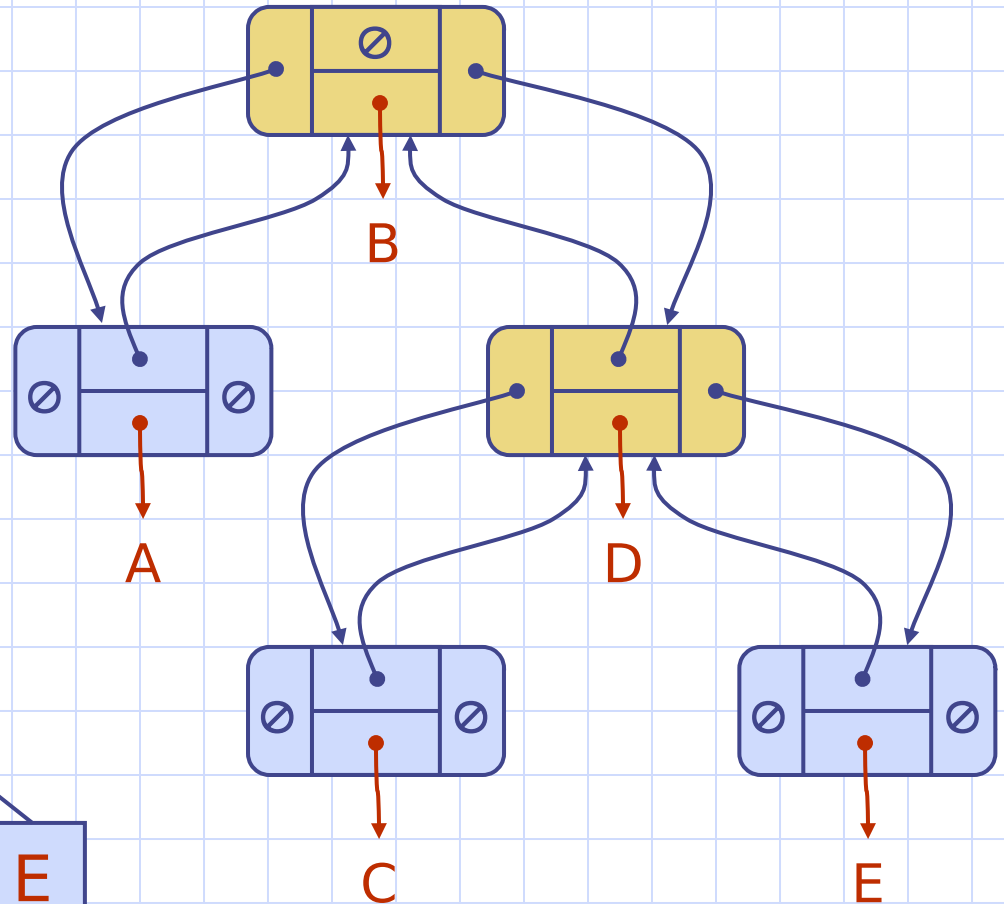
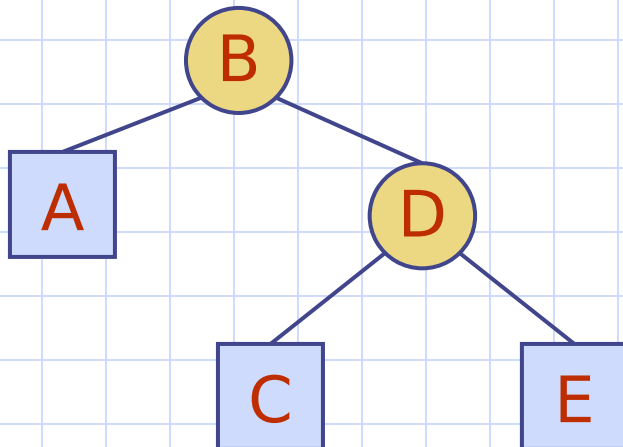
Linked Structure for Trees

- A node is represented by an object storing
 - Element
 - Parent node
 - Sequence of children nodes
- Node objects implement the Position ADT



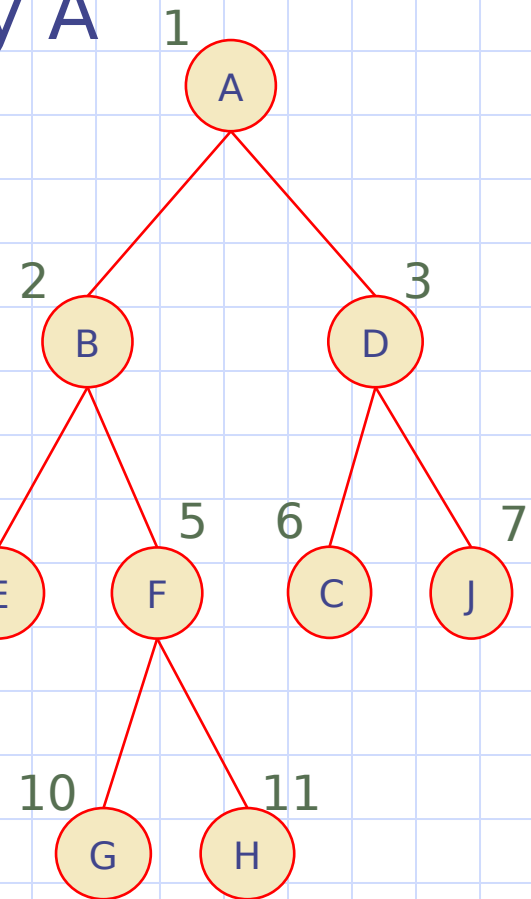
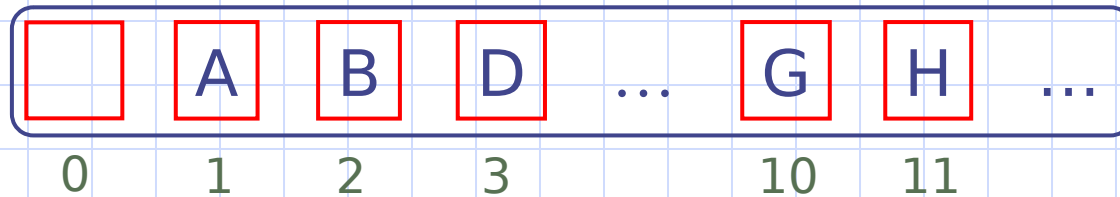
Linked Structure for Binary Trees

- A node is represented by an object storing
 - Element
 - Parent node
 - Left child node
 - Right child node
- Node objects implement the Position ADT



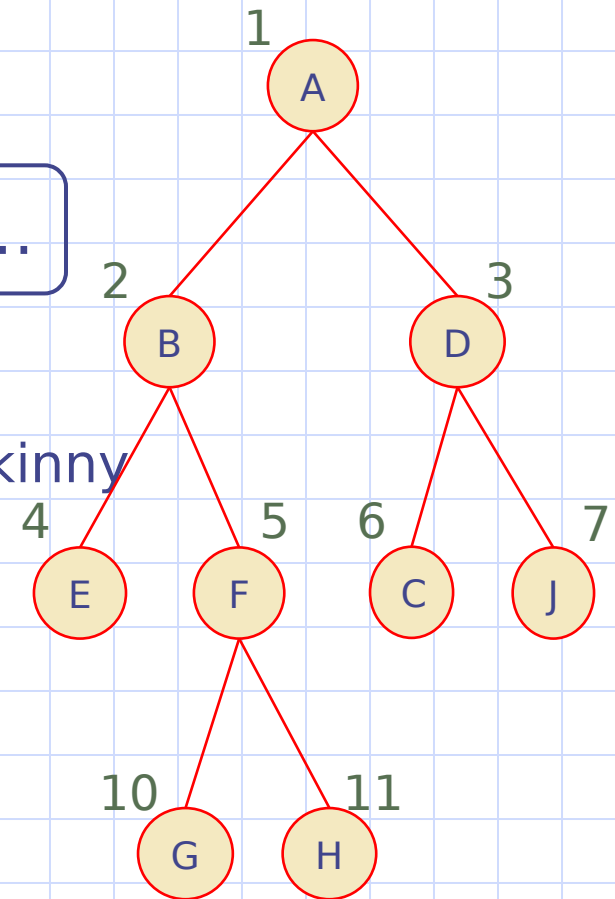
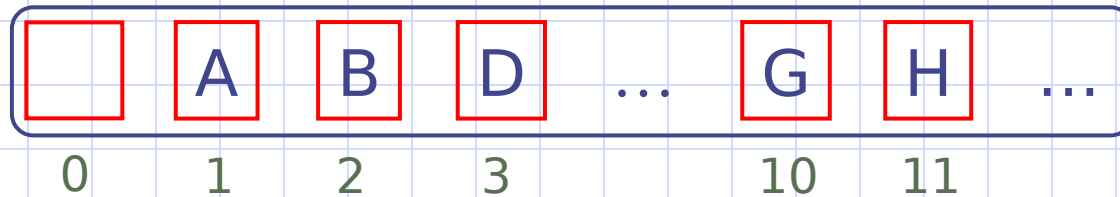
Array-Based Representation of Binary Trees

- Nodes are stored in an array A



- The left child of node k is stored in array location $2k$
- The right child of node k is stored in array location $2k+1$
- The parent of node m is in location $\text{floor}(m/2)$, the largest integer $\leq m/2$

Array-Based Representation of Binary Trees



- ❑ Storage may be wasted if the tree is skinny
- ❑ But access time is as fast as linked structure.